GLOBALRAIN

Practices for Secure Software Report

Table of Contents

Document Revision History

Version	Date	Author	Comments
1.1	6/26/2026	Rex Waymire	Financial Cipher

Client

Developer
Rex Waymire

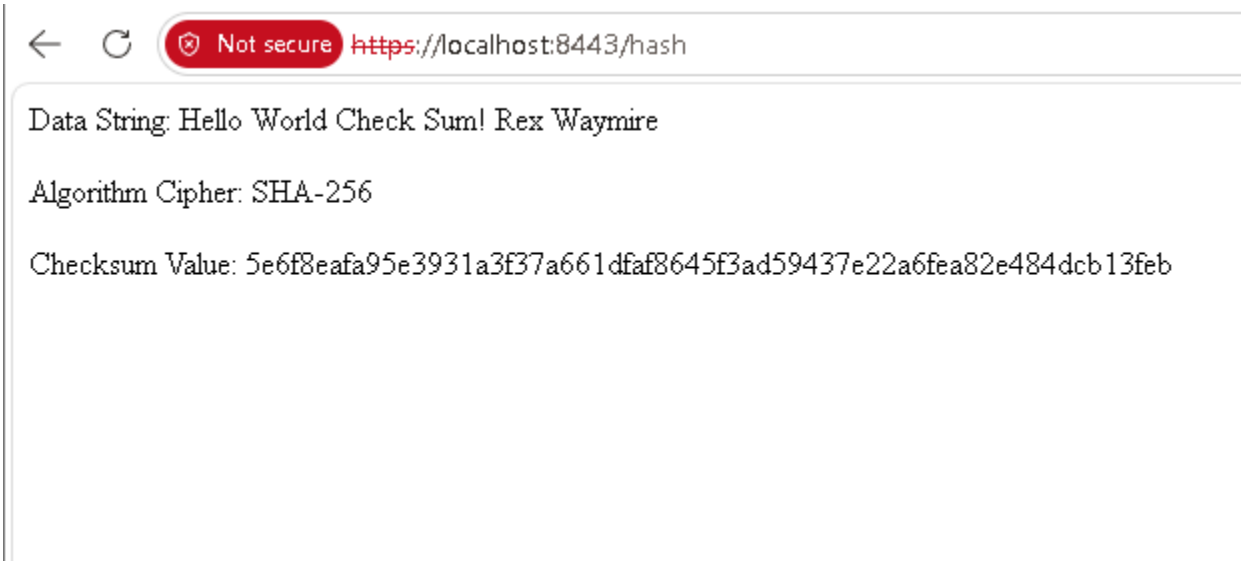
1. Algorithm Cipher

The Algorithmic Cipher that I chose for Artemis Financials case is SHA-256. SHA-256 is part of the SHA-2 group, which is a hash function for case checking. This is standardized by NIST under FIPS 180-4. It produces a 256-bit output regardless of input size. SHA-256 uses one-way hashing, meaning that it can not be decrypted back to the original input. This makes it particularly appropriate for financial checksum verification, rather than encryption and decryption. It does not use keys. It takes an input and processes it through the hash function and returns a fixed-length digest. If any character changes, then the output changes entirely. SHA-256 has no known practical collision attacks. It remains the current industry standard for integrity verification, used in TLS certificates, and code signing.

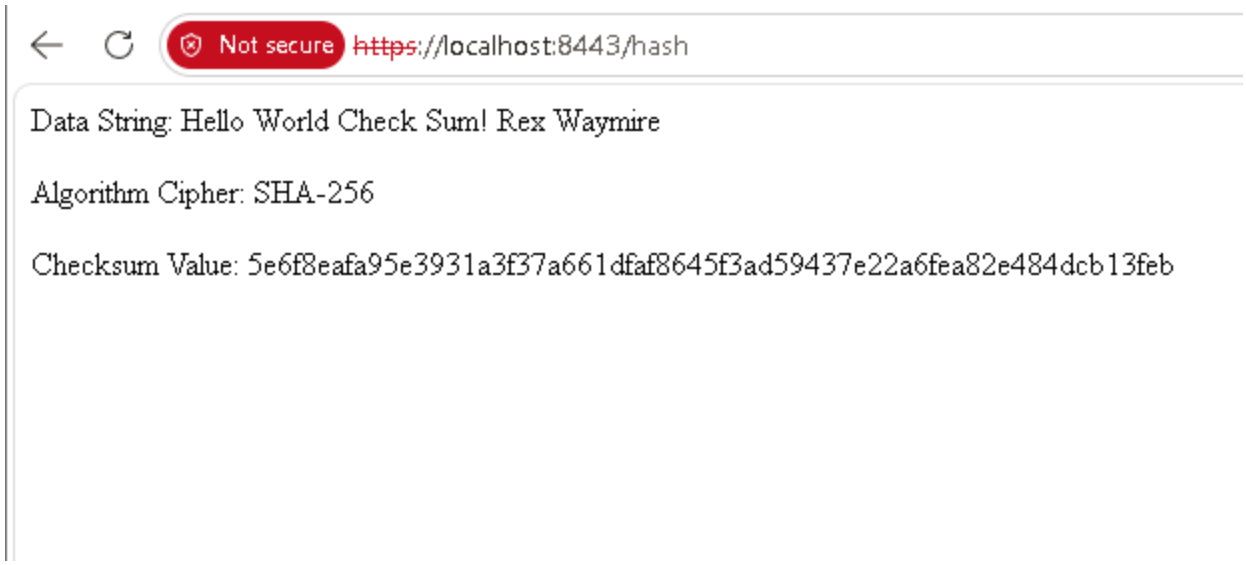
2. Certificate Generation

```
PS C:\Users\waymi> keytool -printcert -file server.cer
Owner: CN=Rex Waymire, OU=CS, O=Global Rain, L=BBL, ST=California, C=US
Issuer: CN=Rex Waymire, OU=CS, O=Global Rain, L=BBL, ST=California, C=US
Serial number: fef75d8b57c409eb
Valid from: Sun Jun 28 17:42:01 PDT 2026 until: Wed Jun 23 17:42:01 PDT 2027
Certificate fingerprints:
         SHA1: A3:25:EB:5D:78:94:EF:5B:B8:69:50:3D:18:90:75:34:79:52:79:9B
         SHA256: FF:64:34:0F:06:C1:86:73:EE:37:51:C0:DC:07:4B:FF:57:B5:7C:0D:A7:91:17:14:9A:EF:1C:7F:AC:16:7A:29
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: C3 77 8F 82 32 60 6D 18   AC 1F 77 77 05 12 1C 48  .w..2`m...ww...H
0010: 34 EB 92 52                                        4..R
]
]
```

3. Deploy Cipher

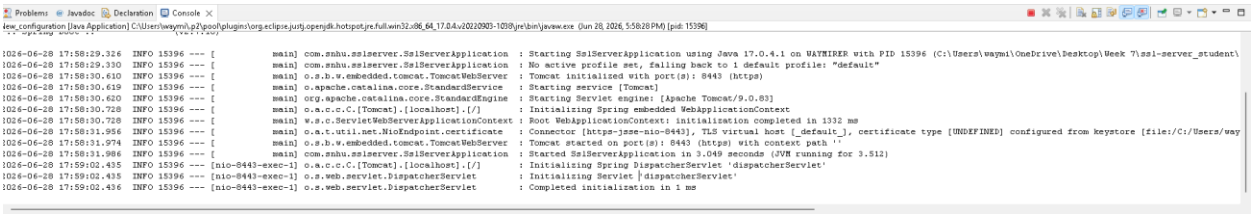


4. Secure Communications



5. Secondary Testing

The dependency check report shows vulnerabilities that are existing in third-party libraries including in the original code base. The refactoring that I did, did not add new vulnerabilities or dependencies.



DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

How to read the report | Suppressing false positives | Getting Help: github issues

Sponsor

Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information (show all):

- *dependency-check version*: 12.1.0
- *Report Generated On*: Sun, 28 Jun 2026 18:04:05 -0700
- *Dependencies Scanned*: 44 (26 unique)
- *Vulnerable Dependencies*: 13
- *Vulnerabilities Found*: 147
- *Vulnerabilities Suppressed*: 0
- ...

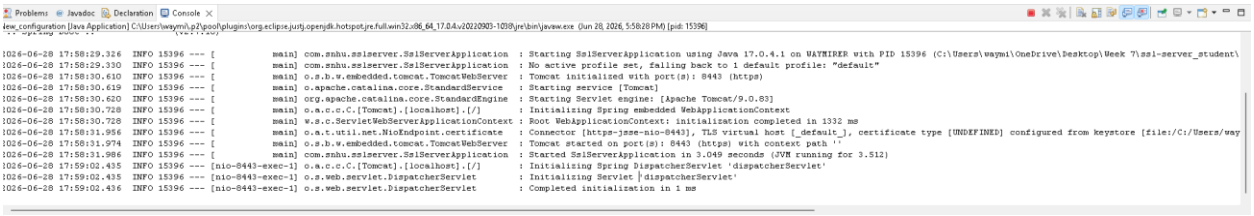
Summary

Display: Showing Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	H
jackson-databind-2.13.5.jar	cpe:2.3:a:fasterxml:jackson-core:2.13.5:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-databind:2.13.5:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.13.5:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.13.5	H
json-path-2.7.0.jar	cpe:2.3:a:json-path:jayway_jsonpath:2.7.0:*:*:*:*:*:*:*	pkg:maven/com.jayway.jsonpath/json-path@2.7.0	M
log4j-api-2.17.2.jar	cpe:2.3:a:apache:log4j:2.17.2:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.17.2	M
logback-core-1.2.12.jar	cpe:2.3:a:qos:logback:1.2.12:*:*:*:*:*:*:*	pkg:maven/ch.qos.logback/logback-core@1.2.12	H
snakeyaml-1.30.jar	cpe:2.3:a:snakeyaml_project:snakeyaml:1.30:*:*:*:*:*:*:*	pkg:maven/org.yaml/snakeyaml@1.30	C
spring-boot-...	cpe:2.3:a:vmware:spring_boot:2.7.18:*:*:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-...	C

6. Functional Testing

I manually reviewed the refactored code to check for any syntactical, logical, or security vulnerabilities. The code runs without errors. Every import that I added has a purpose and it also does not add any new dependencies. The MessageDigest Object initialized with SHA-256 before anything else runs. The bytesToHexfunction handles single-character hex values correctly so that the checksum does not come out wrong. The RESTful route is mapped properly and returns the data string, algorithm name, and checksum to the browser. I did not find any vulnerabilities in the code that I added.



7. Summary

The refactoring that I did on Artemis Financial's application added two main layers of security. The first layer was converting the application from HTTP to HTTPS. I did this by generating a self-signed certificate using the Java Keytool and then configuring the SSL settings in the application properties file. This means that the data sent between the client and the server is now encrypted while it is in transit, and it cannot be read by someone who intercepts the connection.

The second layer was the SHA-256 checksum verification. I implemented this using Java's MessageDigest class. It takes the unique data string that includes my name, runs it through SHA-256, and then returns the hash value to the browser through a RESTful endpoint. A client can use this hash to verify that a file that they downloaded from Artemis Financial has not been changed.

When I look at the vulnerability assessment process flow diagram, the areas that I addressed were secure communication, encryption, and static testing. I ran the OWASP dependency check after refactoring to make sure that I did not introduce any new vulnerabilities. The vulnerabilities that showed up in the report came from libraries that were already in the original project. The refactoring that I did, did not add any new dependencies or make the vulnerability situation any worse.

8. Industry Standard Best Practices

The best practices that I applied during this project follow current industry and NIST standards. I used HTTPS with TLS, which is the current standard for protecting data while it is in transit. The self-signed certificate that I generated uses a 2048-bit RSA key pair, which meets the current minimum recommendation for RSA key length. I also chose SHA-256 over older algorithms like MD5 and SHA-1. Both of those have known collision attacks and are no longer considered secure. SHA-256 has no known practical collision attacks and is approved by NIST under FIPS 180-4.

I also integrated the OWASP dependency check into the Maven build process. This is part of a DevSecOps approach, where the security testing happens during development and not after the software is already deployed. Running it as part of the build means that vulnerabilities in third-party libraries get caught early.

These practices matter for Artemis Financial specifically because the company handles sensitive financial data. If that data is intercepted or tampered with, it could cause serious harm to their clients and also to the company's reputation. Using current encryption standards and running dependency checks regularly

reduces that risk. It also shows clients and regulators that Artemis Financial is taking security seriously, which is important for a company that works in the financial industry.